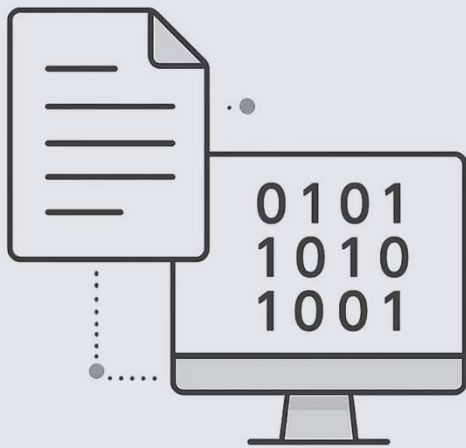




Archivos de Texto y Binario

CONCEPTOS INICIALES

*Una introducción esencial al manejo de archivos en el lenguaje C:
cómo se clasifican, cómo se abren, cómo se leen y cómo se escriben.*

Lic. Jonathan G. Pécora

¿Qué vamos a ver?

En esta presentación recorreremos los conceptos fundamentales del trabajo con archivos en C, desde su definición hasta la implementación de operaciones completas de lectura y escritura.



Leer

Cómo recuperar datos almacenados en un archivo, carácter a carácter o línea a línea, usando funciones estándar de C.



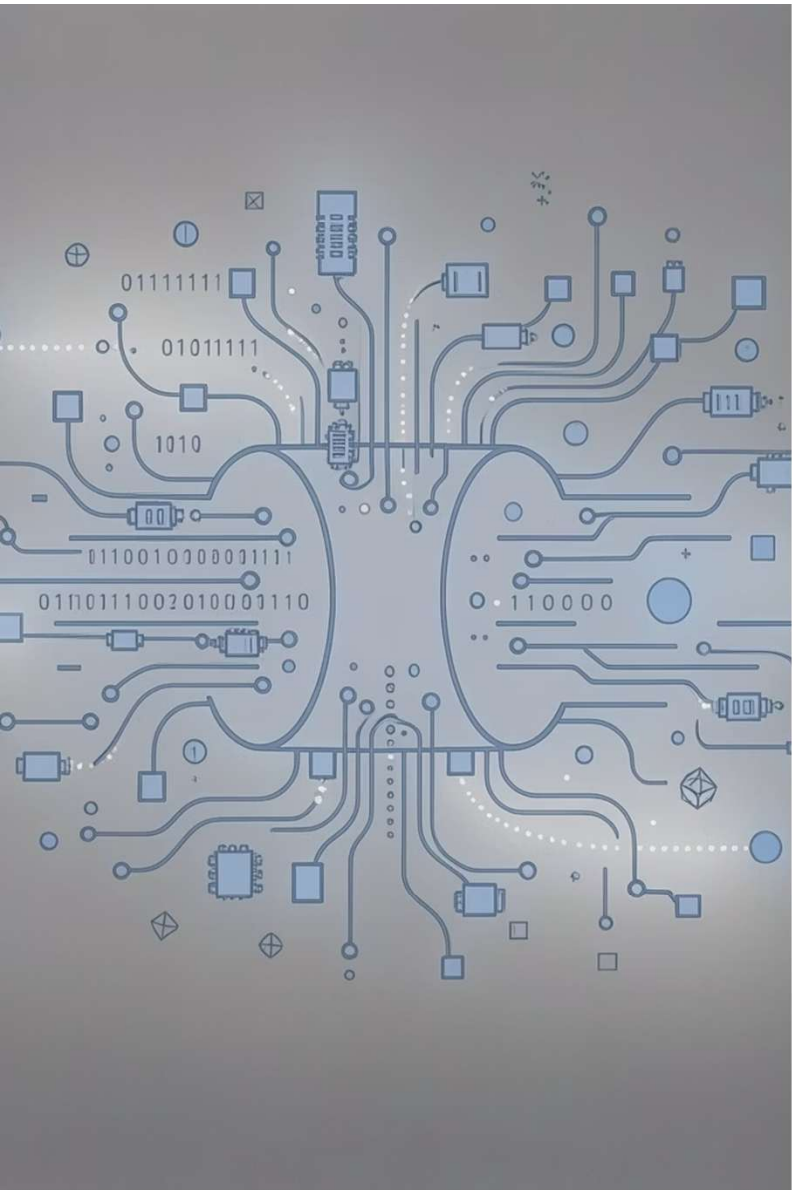
Archivo

Qué es un archivo en el contexto de la programación en C y cómo lo representa el lenguaje mediante la estructura FILE.



Escribir

Cómo almacenar datos en un archivo, tanto en formato de texto como en formato binario, con funciones como `putc`, `fputs` y `fwrite`.



¿Qué es un Archivo en C?

DEFINICIÓN Y ABSTRACCIÓN

Flujo (Stream)

*Un archivo en C se maneja a través de un **flujo o stream**: una corriente de datos que conecta el programa con un dispositivo de almacenamiento externo. Esta abstracción permite tratar de manera uniforme distintos tipos de dispositivos.*

Estandarizado por ANSI C

El manejo de archivos se define en la biblioteca `<stdio.h>`, estandarizada por ANSI C. Toda operación con archivos —lectura, escritura, posicionamiento— se realiza a través del puntero a la estructura `FILE`, que actúa como canal de comunicación.

Estados de un Archivo: Abierto y Cerrado

Un archivo pasa por estados bien definidos durante su ciclo de vida en un programa. Es fundamental comprender la diferencia y respetar el orden correcto de operaciones.

Abierto

El archivo está vinculado al programa mediante un puntero `FILE*`. El stream está activo y se pueden realizar operaciones de lectura o escritura. Se abre con `fopen()`.

Cerrado

*El archivo se desvincula del programa con `fclose()`. Los datos pendientes en el buffer se vuelcan al disco. **No cerrar el archivo puede provocar pérdida de datos.***

 **Siempre cerrar el archivo con `fclose()` al finalizar su uso. Olvidarlo puede corromper los datos almacenados.**

Tipos de Archivos: Clasificación por Acceso

Los archivos pueden clasificarse según la forma en que se accede a sus datos. Cada tipo tiene ventajas según el caso de uso y el volumen de información a gestionar.



Secuenciales

Los registros se leen o escriben en orden, uno tras otro, desde el inicio hasta el fin. Es el tipo más sencillo. Ideal para procesar todos los datos de corrido, como logs o exportaciones.



Acceso Directo

Permite posicionarse en cualquier registro sin recorrer los anteriores. Utiliza funciones como `fseek()` y `ftell()`. Muy eficiente para bases de datos y archivos binarios estructurados.



Indexados

Combinan el acceso secuencial con una tabla de índices que permite localizar registros rápidamente por clave. Son la base de muchos motores de bases de datos modernos.

Archivo Secuencial: Apertura con `fopen()`

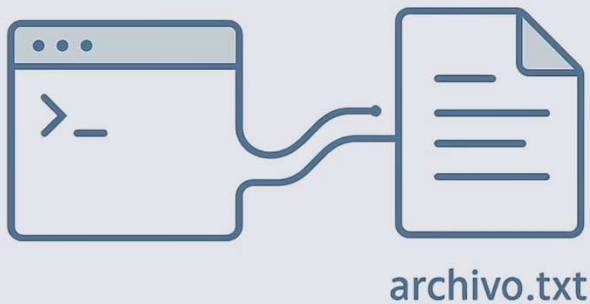
ARCHIVOS DE TEXTO

La función `fopen()` es el punto de entrada para trabajar con cualquier archivo en C. Recibe el nombre del archivo y el modo de apertura, y devuelve un puntero a la estructura `FILE`.

Modos de Apertura

Tabla 15.1 Modos de apertura de un archivo

Modo	Significado
"r"	Abre para lectura.
"w"	Abre para crear nuevo archivo (si ya existe se pierden sus datos).
"a"	Abre para añadir al final.
"r+"	Abre archivo ya existente para modificar (leer/escribir).
"w+"	Crea un archivo para escribir/leer (si ya existe se pierden los datos).
"a+"	Abre el archivo para modificar (escribir/leer) al final. Si no existe es como w+.



Archivo Secuencial: El Puntero FILE*

APERTURA — DETALLES

Al abrir un archivo, declaramos un puntero de tipo `FILE*` que actúa como manejador (handle) del stream. Si `fopen()` falla, devuelve `NULL`, lo que debe verificarse siempre.

```
FILE *archivo;  
char ch;  
char* nombre = "texto99.txt"; // Nombre físico del archivo  
  
if ((archivo = fopen(nombre, "w")) == NULL)  
    printf("*** El archivo %s no pudo abrirse ***\n", nombre);
```

`fopen()`

Función para abrir el archivo y obtener su puntero.

Modo "w"

Indica que el archivo se abre para escritura de texto.

`NULL`

Valor devuelto si el archivo no pudo abrirse. Siempre verificar.

Archivo Secuencial: Escritura – Implementación Completa

EJEMPLO DE CÓDIGO

Este programa solicita caracteres al usuario y los almacena en un archivo de texto usando `putc()`, hasta que el usuario presiona **Enter**. Al finalizar, cierra correctamente el archivo con `fclose()`.

```
#include <stdio.h>
#include <conio.h>
#define p printf

int main(){
    FILE *archivo;
    char ch;
    char* nombre = "texto99.txt";

    if((archivo = fopen(nombre, "w")) == NULL)
        p("\n\n*** El archivo %s no pudo abrirse ***\n", nombre);
    else {
        p("\n\n Ingrese caracteres hasta [Enter] ");
        while((ch = getchar()) != '\n')
            putc(ch, archivo); // Almacena en el archivo hasta [ENTER]
        fclose(archivo);
    }
}
```

 `putc(ch, archivo)` escribe un único carácter en el stream apuntado por `archivo`. El bucle termina al detectar el carácter de salto de línea `'\n'`.

Archivo Secuencial: Lectura Carácter a Carácter

EJEMPLO DE CÓDIGO

Este programa lee un archivo de texto carácter a carácter usando `getc()` y los muestra en pantalla.

El bucle continúa hasta encontrar EOF (End Of File), que marca el final del archivo.

```
#include <stdio.h>
#include <conio.h>
#define p printf

int main(){
    FILE *archivo;
    char ch;
    char* nombre = "texto99.txt";

    if((archivo = fopen(nombre, "r")) == NULL)
        p("\n\n*** El archivo %s no pudo abrirse ***\n", nombre);
    else {
        p("\n\n Contenido del archivo %s caracter a caracter\n\n", nombre);
        while((ch = getc(archivo)) != EOF)
            p("%c", ch);
        fclose(archivo);
    }
}
```

 EOF es una constante definida en `<stdio.h>` que indica el fin del archivo. `getc()` devuelve EOF cuando no hay más datos para leer.

Archivo Secuencial: Lectura por Líneas con `fgets()`

EJEMPLO DE CÓDIGO

Una alternativa más práctica que leer carácter a carácter es leer línea completa con `fgets()`. Esta función almacena hasta `n-1` caracteres en una cadena, deteniéndose ante un salto de línea o el fin del archivo.

```
#include <stdio.h>
#include <conio.h>

int main(){
    FILE *archivo;
    char cadena[81];
    char* nombre = "F:\\Textos\\texto91.txt";

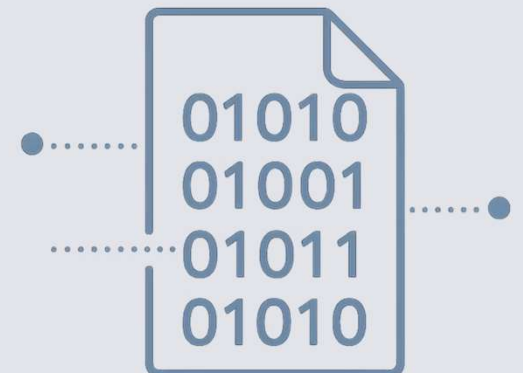
    if((archivo = fopen(nombre, "r")) == NULL)
        printf("\n\n*** El archivo %s no pudo abrirse ***\n", nombre);
    else {
        while(fgets(cadena, 81, archivo)) // Lee hasta \n o n-1 caracteres
            printf("%s", cadena);
        printf("\n\n**FIN**\n");
        fclose(archivo);
    }
    return 0;
}
```

✓ `fgets()` es más segura que `gets()` porque limita la cantidad de caracteres leídos, evitando desbordamientos de buffer.

Archivo Binario: Acciones Posibles

Los archivos binarios permiten almacenar datos tal como están representados en memoria —structs, enteros, floats— sin conversión a texto.

Las operaciones fundamentales son abrir, escribir con `fwrite`, leer con `fread`, posicionar con `fseek` y cerrar con `fclose`.



Archivo Binario: Apertura

MODO BINARIO

La apertura de un archivo binario es idéntica a la de texto, con una diferencia clave: se agrega la letra **"b"** al modo de apertura. Esto indica al sistema operativo que el archivo debe tratarse sin conversión de caracteres especiales.

"rb"

Abre para lectura binaria.

"wb"

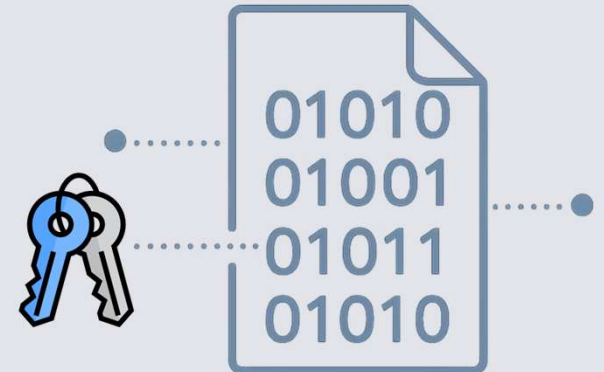
Abre para escritura binaria (crea o trunca).

"ab"

Abre para añadir al final en binario.

"rb+"

Lectura y escritura binaria sobre archivo existente.



Archivo Binario: Ejemplo de Apertura en Código

MODO BINARIO — IMPLEMENTACIÓN

A continuación se muestra cómo declarar el puntero, definir la ruta del archivo y abrirlo en modo binario de lectura/escritura. Observar la presencia de **"rb+"** como modo.

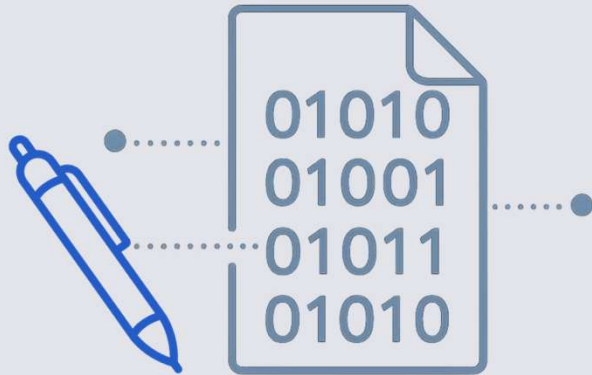
```
FILE *cli;  
char archivo[] = "F:\\ARCHIVOS_PRUEBA\\clientes.dat";  
  
system("cls");  
  
if ((cli = fopen(archivo, "rb+")) == NULL)  
    printf("Error: el archivo no pudo abrirse.\n");
```

La diferencia clave

La **"b"** en el modo le indica a C que los datos deben leerse/escribirse tal cual están en memoria, sin interpretación de saltos de línea ni otros caracteres de control.

Verificar siempre NULL

Si `fopen()` devuelve `NULL`, el archivo no pudo abrirse. Ignorar este chequeo puede generar accesos inválidos a memoria y comportamientos impredecibles en el programa.



Archivo Binario: Función `fwrite()`

La función `fwrite()` escribe bloques de datos en un archivo binario. Es la herramienta principal para almacenar estructuras completas en disco de forma eficiente y sin conversión.

```
fwrite(&c, sizeof(c), 1, x);
```

`&c`

Dirección de memoria de la variable o estructura a escribir.

`sizeof(c)`

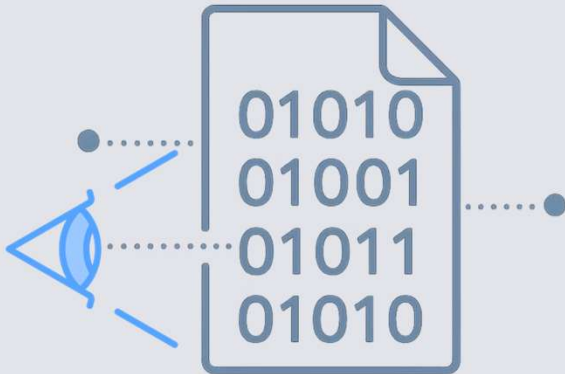
Tamaño en bytes del dato. Garantiza que se escriban todos los campos.

`1`

Cantidad de elementos a escribir en esta llamada.

`x`

Puntero `FILE*` del archivo de destino.



Archivo Binario: Función `fread()`

La función `fread()` es la contraparte de `fwrite()`: lee bloques de datos desde un archivo binario y los carga en una variable o estructura en memoria. Su firma es simétrica.

```
fread(&c, sizeof(c), 1, x);
```

`&c`

Dirección donde se almacenará el dato leído del archivo.

`sizeof(c)`

Tamaño en bytes del elemento a leer.

1

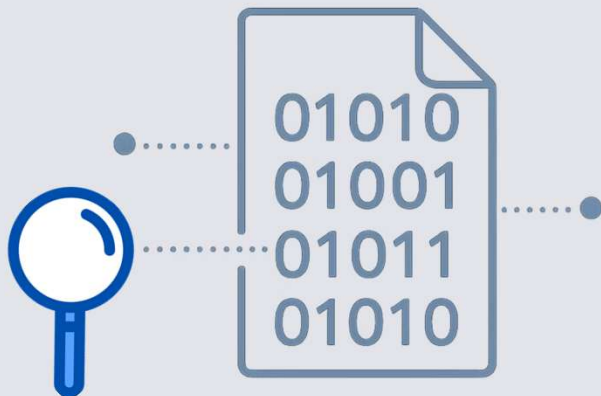
Número de elementos a leer por llamada.

`x`

Puntero `FILE*` del archivo fuente.



Tras cada llamada a `fread()`, verificar con `feof()` si se llegó al final del archivo para evitar procesar datos inválidos.



Archivo Binario: Función `fseek()`

La función `fseek()` permite posicionar el cursor de lectura/escritura en cualquier punto del archivo, habilitando el **acceso directo** a registros específicos sin necesidad de recorrer el archivo secuencialmente.

```
fseek(x, 0L, SEEK_SET);
```

SEEK_SET

*Posiciona el cursor desde el **inicio** del archivo.*

SEEK_CUR

*Posiciona el cursor desde la **posición actual**.*

SEEK_END

*Posiciona el cursor desde el **final** del archivo.*

Ejemplo: Carga en Archivo Binario – Parte 1 de 3

CASO COMPLETO

Este ejemplo muestra cómo definir una estructura `Registro` con nombre y edad, y abrir un archivo binario en modo **"ab"** (append binary) para agregar registros al final sin perder los existentes.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

typedef struct {
    char nombre[25];
    int edad;
} Registro;

int main(){
    Registro persona;
    FILE *archivo;

    archivo = fopen("datos.dat", "ab");
    // "ab" = append binario: agrega al final sin borrar datos previos
```

 El modo **"ab"** (append binary) es ideal cuando se desea agregar nuevos registros a un archivo sin sobrescribir los anteriores.

Ejemplo: Carga en Archivo Binario – Parte 2 de 3

CASO COMPLETO

El bucle `do-while` solicita nombre y edad repetidamente. Cada registro se escribe con `fwrite()`, pasando la dirección de la estructura, su tamaño, la cantidad de registros (1) y el puntero al archivo.

```
if(archivo != NULL){
do {
fflush(stdin); /* Vacía el buffer de teclado */
printf("\nIntroduzca el nombre: ");
scanf("%s", persona.nombre);

if(strlen(persona.nombre) > 0){
printf("Introduzca la edad: ");
scanf("%d", &(persona.edad));

// Dir. variable | tamaño struct | 1 registro | archivo
fwrite(&persona, sizeof(persona), 1, archivo);

printf("\nDesea continuar (s/n): ");
fflush(stdin);
tecla = getchar();
}
} while(tecla == 's');
```

Ejemplo: Carga en Archivo Binario – Parte 3 de 3

CASO COMPLETO

Si el archivo no pudo abrirse, se informa el error. Al finalizar la carga, siempre se cierra el archivo para garantizar que todos los datos se escriban correctamente en disco.

```
    } else {  
        printf("Error en la apertura del archivo");  
    }  
  
    fclose(archivo); // ¡Siempre cerrar el archivo!  
    system("pause");  
    return 0;  
}
```



Verificar apertura

Siempre comprobar que `archivo != NULL` antes de operar.



Escribir con `fwrite`

Pasar dirección, tamaño, cantidad y puntero `FILE*` correctamente.



Cerrar con `fclose`

Libera el recurso y vuelca el buffer al disco sin pérdidas.

Ejemplo: Lectura de Archivo Binario – Parte 1 de 2

CASO COMPLETO

Para leer un archivo binario con registros de tipo `Registro`, se declara la misma estructura usada para escribir y se abre el archivo en modo **"rb"** (read binary).

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

typedef struct {
    char nombre[25];
    int edad;
} Registro;

int main(){
    Registro persona;
    FILE *archivo;

    archivo = fopen("datos.dat", "rb");
    // "rb" = lectura binaria: no modifica el archivo
```



La estructura utilizada para leer **debe coincidir exactamente** en campos y tipos con la usada al escribir. De lo contrario, los datos leídos serán incorrectos.

Ejemplo: Lectura de Archivo Binario – Parte 2 de 2

CASO COMPLETO

Se utiliza la técnica de **lectura adelantada**: se lee un registro antes del bucle y dentro del bucle se procesa primero, luego se lee el siguiente. Esto evita que el último registro se procese dos veces.

```
if(archivo != NULL){
    printf("Registros (Nombre y Edad) del archivo binario\n\n");

    fread(&persona, sizeof(Registro), 1, archivo); // Lectura adelantada

    while(!feof(archivo)){
        printf("Nombre: %s, Edad: %d\n", persona.nombre, persona.edad);
        fread(&persona, sizeof(Registro), 1, archivo);
    }

    fclose(archivo);
} else {
    printf("Error en la apertura del archivo");
}

system("pause");
return 0;
}
```

Ejemplo: Conteo de Registros – Parte 1 de 2

CASO COMPLETO

Una operación muy útil en archivos binarios es contar cuántos registros contiene el archivo sin leerlos todos. Esto se logra combinando `fseek()` y `ftell()`.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

typedef struct {
    char nombre[25];
    int edad;
} Registro;

int main(){
    int cantReg;
    Registro persona;
    FILE *archivo;

    archivo = fopen("datos.dat", "rb");
    // Abrimos solo para lectura binaria
```

 No necesitamos leer cada registro para saber cuántos hay. Podemos calcular la cantidad a partir del tamaño total del archivo dividido por el tamaño de cada registro.

Ejemplo: Conteo de Registros – Parte 2 de 2

CASO COMPLETO

La fórmula es sencilla y elegante: posicionarse al final del archivo con `fseek(SEEK_END)`, obtener el byte actual con `ftell()` y dividir entre el tamaño de un registro.

```
if(archivo != NULL){
    fseek(archivo, 0, SEEK_END); // Ir al final del archivo

    cantReg = ftell(archivo) / sizeof(Registro);
    // ftell() devuelve la posición actual = tamaño total en bytes

    printf("\nCantidad de registros en el archivo = %d\n", cantReg);
} else {
    printf("Error en la apertura del archivo\n");
}

system("pause");
return 0;
}
```

`fseek` → `SEEK_END`

Mueve el cursor al final del archivo.

`ftell()`

Devuelve la posición actual en bytes = tamaño del archivo.

`÷ sizeof(Registro)`

Divide para obtener el número exacto de registros.



¿Consultas?

*Esta presentación cubrió los conceptos fundamentales del manejo de archivos de texto y binarios en C:
flujos, modos de apertura, funciones de lectura y escritura, y ejemplos completos con estructuras. ¡No dudes en preguntar!*

Lic. Jonathan G. Pécora

Instituto Nacional Superior del Profesorado Técnico — UTN